02 / ALGEBRA / EX25

EX25 exercise guide

Part A: pure algebra · Part B: forces represented as vectors.

DOCUMENT 03 / EXERCISE

What you will learn

- Verify vector-algebra calculations manually.
- Represent forces using basis vectors and ordered components.
- Write your own NumPy script and check its output.

Prerequisites: EX25 theory and reference guides. No JSON, pytest, or complex project structure required.

Reading order: 1. Theory → 2. Reference → 3. Exercise → 4. Pseudocode → 5. Programming.

1. Starting point and goal

EX25 has **two parts**. Part A confirms that you understand vector operations before introducing physics. Part B applies exactly the same operations to two forces. First work the mathematics on paper; then reproduce it in Python.

2. Part A — Fully solved algebra example

Given: dimensionless vectors.

$$\mathbf{u} = (3, -1), \qquad \mathbf{v} = (-2, 4)$$

A1. Addition:

$$\mathbf{u} + \mathbf{v} = (3 - 2, \ -1 + 4)$$

$$\mathbf{u} + \mathbf{v} = (1, 3)$$

A2. Linear combination:

$$2\mathbf{u} - \mathbf{v} = (6, -2) - (-2, 4)$$

$$2\mathbf{u} - \mathbf{v} = (8, -6)$$

A3. Norms:

$$\|\mathbf{u} + \mathbf{v}\|_2 = \sqrt{10} \approx 3.1623$$

$$\|2\mathbf{u} - \mathbf{v}\|_2 = \sqrt{100} = 10$$

Your task: reproduce these results in Python and check them against the manual steps. Then work independently with:

$$\mathbf{a} = (2, -1), \qquad \mathbf{b} = (-3, 5)$$

Compute **a + b**, **2a - b**, and the corresponding norms. Check values:

$$\mathbf{a} + \mathbf{b} = (-1, 4), \qquad 2\mathbf{a} - \mathbf{b} = (7, -7)$$

$$\|\mathbf{a} + \mathbf{b}\|_2 = \sqrt{17} \approx 4.1231$$

$$\|2\mathbf{a} - \mathbf{b}\|_2 = \sqrt{98} \approx 9.8995$$

3. Part B — Engineering problem: forces in 2D

Two forces act on a point in a two-dimensional Cartesian coordinate system. Represent their components using the unit vectors along the axes:

$$\hat{\mathbf{i}} = (1, 0), \qquad \hat{\mathbf{j}} = (0, 1)$$

Given forces:

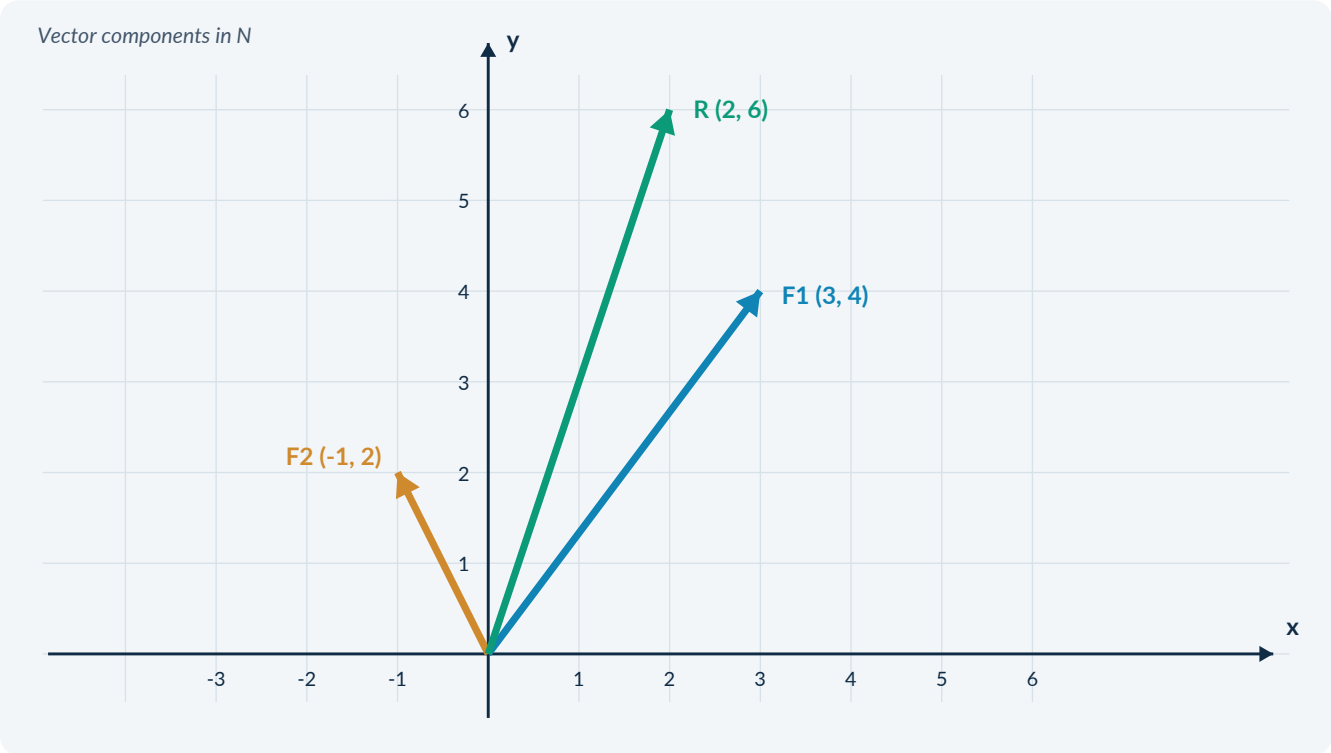
$$\mathbf{F}_1 = (3\hat{\mathbf{i}} + 4\hat{\mathbf{j}})\ \mathrm{N}$$

$$\mathbf{F}_2 = (-1\hat{\mathbf{i}} + 2\hat{\mathbf{j}})\ \mathrm{N}$$

Symbols **î** and **ĵ** are Cartesian **unit directions**, not imaginary numbers. The coefficients 3, 4, -1, and 2 are force components in newtons.

The equivalent ordered-component form is:

$$\mathbf{F}_1 = (3, 4)\ \mathrm{N}, \qquad \mathbf{F}_2 = (-1, 2)\ \mathrm{N}$$



4. What to calculate

- Create F1 and F2 as NumPy arrays.
- Calculate each force magnitude separately.
- Calculate the resultant vector R = F1 + F2.
- Calculate the magnitude of the resultant.
- Print labeled results with units and at least three decimal places.
- Manually check the results before accepting the program.

5. Reference results (not the solution code)

Quantity	Expected value
F1 components	(3, 4) N
F2 components	(-1, 2) N
Magnitude of F1	5.000 N
Magnitude of F2	2.236 N (approx.)
Resultant vector	(2, 6) N
Resultant magnitude	6.325 N (approx.)

6. Implementation requirements

Exercise location and folder structure:

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors/
            ├── EX25_vectors_and_forces.py
            └── docs/
                ├── ES/
                └── EN/
```

File to implement: `EX25_vectors_and_forces.py`. Use **a single Python script** for now. The **docs** folder holds Spanish (ES) and English (EN) guides. No data files, src folder, JSON, or tests are required.

Run it from your VS Code terminal with `python EX25_vectors_and_forces.py`. Use comments to distinguish parts A and B.

7. Pseudocode guidance

Before writing code, outline the logic on paper: define inputs → create arrays → perform vector operations → calculate norms → print → compare with manually calculated results. The **Pseudocode Guide** develops this process.

8. Coding hints (placed at the end)

These lines illustrate only the necessary syntax. **They are not the full solution**.

```
import numpy as np

example = np.array([6.0, 8.0])
length = np.linalg.norm(example)
print(f"Example magnitude: {length:.3f}")
```

Remember: NumPy arrays support componentwise addition, subtraction, and multiplication. Review the reference guide to see each operation.